上外附中 · 信息研修班 · C++ 习题课　2026 年 10 月

国庆模拟测试赛 · 习题讲评

三场比赛 · 十二道题 · 每题三讲：题目 → 思路 → 参考代码

模拟测试赛一 · 四题　模拟测试赛二 · 四题　模拟测试赛三 · 四题

12

道题，逐一击破

教学设计
讲评课 · 1 课时

课堂结构
每题 5 分钟 + 共性总结 15 分钟

核心目标
从订正一道题，到会做一类题

教学设计总览

本课件对应三场国庆模拟测试赛的合卷讲评，共 12 题。每道题固定三页：题目回顾、思路拆解、参考代码，最后用 5 页归纳共性规律。

学情说明

研修班学生已掌握变量、分支、循环、数组、字符串与结构体排序，尚未系统学习算法。本课定位：用 12 道“暴力可过”的真题，把模拟与枚举的基本功打扎实。

教学目标

- 能独立订正 12 道题，说清每题“枚举什么、统计什么”。
- 掌握打擂台、桶计数、拆位、双关键字排序、整词匹配五个基本套路。
- 记住解题四步法，并会用“数据范围”反推可行的算法。

教学重点

- 把题意准确翻译成循环与判断（翻译能力）。
- 桶 / 标记数组的思想与使用场景。
- 边界与特判：0、负数、并列、等号、格式。

教学难点

- 连续正整数和的“起点上界 + 剪枝”。
- 回文日期“构造另一半”的转化思维。
- 统计单词数的整行读入与整词匹配细节。

课后任务

- 未 AC 的题按三页顺序重新订正。
- 把 12 题按“五大武器”归类，写进错题本。
- 任选一题，把思路讲给同桌听。

课堂流程 每题（5 分钟）：题目回顾 1 分钟 → 思路引导 2 分钟 → 代码精讲 2 分钟；12 题讲完后：共性规律总结 15 分钟 → 布置订正任务 5 分钟

目录

CONTENTS

每一道题按「题目 → 思路 → 参考代码」三页展开，最后归纳共性规律。

课前自查：这三场比赛，你各 AC 了几题？

01 **模拟测试赛一** P04 – P16
津津 · 骰子和 · 珠心算 · 分数线 —— 打擂台 / 桶计数 / 数对判定 / 双关键字排序

02 **模拟测试赛二** P17 – P29
小鱼游泳 · 数字统计 · 连续正整数和 · 彩票摇奖 —— 单位换算 / 拆位 / 累加剪枝 / 桶标记

03 **模拟测试赛三** P30 – P42
数字反转 · 多项式输出 · 回文日期 · 统计单词数 —— 拆位重装 / 格式模拟 / 构造 / 整词匹配

04 **共性规律总结** P43 – P49
十二题家谱 · 解题四步法 · 数据范围定算法 · 五大武器 · 考场检查清单

01

模拟测试赛一

P04 – P16 · 每题三讲：题目 / 思路 / 参考代码

A **不高兴的津津**
NOIP 2004 普及组 · 入门 · 循环累加 / 打擂台求最值 / 特判

B **Bovine Bones G**
USACO 2008 OCT · 入门 · 枚举组合 / 桶计数 / 并列取最小

C **珠心算测验**
NOIP 2014 普及组 · 普及– · 枚举 / 数对判定 / 计数

D **分数线划定**
NOIP 2009 普及组 · 普及– · 结构体排序 / 双关键字 / 模拟

不高兴的津津

题目描述

津津每天在学校上课 **a 小时**，妈妈又安排她上 **b 小时**复习班。一天总学习**超过 8 小时**她就会不高兴，且上得越久越不高兴。请检查她周一至周日的安排：她会不高兴吗？哪天最不高兴？

输入 共 7 行，第 i 行两个小于 8 的非负整数 a、b：周 i 在校上课时间与妈妈安排她上课的时间。

输出 一个数字：不会不高兴输出 **0**；否则输出最不高兴的是周几（1~7 表示周一至周日）。程度相当时输出最靠前的一天。

样例

输入

```
5 3
6 2
7 2
5 3
5 4
0 4
0 6
```

输出

```
3
```

数据范围

每个数均为小于 8 的非负整数；时限 1s。

循环累加　打擂台求最值　特判

时间复杂度：**O(7)**

思路 · 七天打擂台，一天一个和

一句话思路：七天循环 + 打擂台求最大值，一天就是一个“和”。

思路拆解

1 把每天翻译成一个数

当天总学习时长 = 在校时间 + 课外时间，即 s = a + b，7 天就是 7 次求和。

2 循环打擂台求最大

用 maxHour 记录当前最大值，s 严格大于才更新——并列时自动保留最靠前的一天。

3 别忘输出分支

打完擂台先判断：maxHour > 8 才输出星期几，否则输出 0。

用样例走一遍

每天求和：**8、8、9、8、9、4、6**

最大值 9 首次出现在**周三**（周五也是 9，但更靠后）

9 > 8 → 输出 **3**

易错提醒

- “超过 8 小时”是严格大于，等于 8 不算不高兴。
- 并列时取最靠前的一天，比较要用 > 而不是 ≥。
- 都不超过 8 小时时输出 0，不是输出最大值。

参考代码 · 不高兴的津津

时间复杂度 O(7)

```cpp
#include <iostream>
using namespace std;

int main() {
    int maxHour = 0, maxDay = 0;        // 最大不高兴值、对应星期
    for (int i = 1; i <= 7; i++) {
        int a, b;
        cin >> a >> b;                  // 上学时间 + 妈妈安排的时间
        int s = a + b;                  // 当天总学习时长
        if (s > maxHour) {              // 严格大于：并列时保留最靠前的一天
            maxHour = s;
            maxDay = i;
        }
    }
    if (maxHour > 8) cout << maxDay << endl;
    else cout << 0 << endl;             // 每天都不超过 8 小时
    return 0;
}
```

代码讲解

1. 打擂台标准写法：maxHour 初值 0，s 严格大于才更新，并列自动取最靠前的一天。
2. s = a + b 就是“把题意翻译成算式”的那一行。
3. 输出前必须判断 maxHour 是否大于 8，两个分支都不能漏。

变式一问

如果题目改为“并列时输出编号最大的那一天”，第 9 行的比较符应该怎么改?

Bovine Bones G

题目描述

Bessie 买了三个骰子，面数分别为 **s1、s2、s3**。她不断同时投掷，想知道：哪个**点数和**出现得最频繁？若有多个和并列最多，输出其中**最小的和**。

输入 一行三个整数 s1、s2、s3（1≤s1、s2≤20，1≤s3≤40）。

输出 一个整数：出现次数最多的最小点数和。

样例

输入

```
3 2 3
```

输出

```
5
```

数据范围

s1、s2 ≤ 20，s3 ≤ 40，全部组合 ≤ 16000 种；时限 1s。

枚举组合 **桶计数** **并列取最小**

时间复杂度：**O(s1•s2•s3) ≤ 1.6万**

思路 · 哪个桶最高?

一句话思路：把“哪种和最常见”变成“哪个桶最高”。

思路拆解

1 确定枚举对象

三个骰子的所有组合，共 s1×s2×s3 ≤ 16000 种，直接三重循环枚举。

2 用桶做统计

开一个桶数组 cnt，每枚举出一组 (i, j, k)，就 cnt[i+j+k]++。

3 从小到大扫桶

和只可能从 3 到 s1+s2+s3；从小到大扫、严格大于才更新，并列时自然取最小的和。

用样例走一遍

s1=3, s2=2, s3=3：共 18 种组合

频次：3:1 · 4:3 · **5:5** · **6:5** · 7:3 · 8:1

5 和 6 并列最多 → 取更小的 **5**

易错提醒

- 三重循环 = 枚举所有组合，共 s1×s2×s3 ≤ 16000 次。
- cnt[i + j + k]++ 一行完成“求和 + 计数”。
- 扫桶从小到大 + 严格大于更新，并列自动取最小的和。

参考代码 · Bovine Bones G

时间复杂度 O(s1·s2·s3) ≤ 1.6万

```cpp
#include <iostream>
using namespace std;

int cnt[85];                    // 桶：cnt[s] = 点数和为 s 的组合数

int main() {
    int s1, s2, s3;
    cin >> s1 >> s2 >> s3;
    for (int i = 1; i <= s1; i++)
        for (int j = 1; j <= s2; j++)
            for (int k = 1; k <= s3; k++)
                cnt[i + j + k]++;    // 枚举全部组合，落入桶中
    int best = 3;                   // 最小可能的和是 3
    for (int s = 4; s <= s1 + s2 + s3; s++)
        if (cnt[s] > cnt[best]) best = s;   // 严格大于 → 并列取最小
    cout << best << endl;
    return 0;
}
```

代码讲解

1. 桶数组开 85 就够：最大和只有 20+20+40=80。
2. 并列时取“最小的和”，不是取“频次”。
3. 扫桶从 3 开始，别从 0 或 1 开始（和最小是 3）。

变式一问

如果变成四个骰子，循环层数和桶的大小分别要怎么改?

珠心算测验

题目描述

老师随机生成一个由**互不相同的正整数**组成的集合。问：其中有多少个数，恰好等于集合中**另外两个（不同的）数**之和?

输入 第一行整数 n（n≤100）；第二行 n 个互不相同的正整数（每个不超过 10000）。

输出 一个整数：满足条件的数的个数。

样例

输入

```
4
1 2 3 4
```

输出

```
2
```

数据范围

n ≤ 100，数值 ≤ 10^4；时限 1s。

枚举　数对判定　计数

时间复杂度：**$O(n^3)$，n≤100**

思路 · 每个数都问一句：能被拆吗？

一句话思路：对每个数问一句：你能被拆成集合里两个不同的数之和吗？

思路拆解

1 确定枚举对象

依次检查集合中的每个数 x，看它能否被“拆”出来。

2 枚举数对做判定

再双重循环枚举两个加数 a[j]、a[k]（j ≠ k），判断 a[j] + a[k] 是否等于 x。

3 放心用暴力

n ≤ 100，数对约 5000 个，总运算量约 10^6，三重循环完全可行。

用样例走一遍

集合 {1, 2, 3, 4}：

3 = **1+2** ✓　4 = **1+3** ✓

2 找不到两个不同的加数 ✗ → 答案 **2**

易错提醒

- k 从 j+1 开始，保证两个加数是不同的数。
- 条件里加 !ok：找到答案立即退出，省一点时间。
- n ≤ 100 时三重循环约 10^6 次，完全可行。

参考代码 · 珠心算测验

时间复杂度 $O(n^3)$，$n \le 100$

```cpp
#include <iostream>
using namespace std;

int main() {
    int n, a[105];
    cin >> n;
    for (int i = 0; i < n; i++) cin >> a[i];

    int ans = 0;
    for (int i = 0; i < n; i++) {    // 逐个检查集合中的数
        bool ok = false;
        for (int j = 0; j < n && !ok; j++)
            for (int k = j + 1; k < n && !ok; k++)
                if (a[j] + a[k] == a[i])   // 两个不同的数之和
                    ok = true;
        if (ok) ans++;
    }
    cout << ans << endl;
    return 0;
}
```

代码讲解

1. “两个不同的数”意味着两个加数不能是同一个，枚举时 k 从 j+1 开始。
2. 加数不会等于和本身：都是正数，两数之和一定大于每个加数。
3. n=100 时三重循环约 10^6 次，别怕暴力，怕的是把“不同”看漏。

变式一问

如果 n 扩大到 5000，三重循环还过得去吗？可以怎样借助“标记数组”加速？

分数线划定

题目描述

面试分数线 = 所有选手按成绩**从高到低**排序后，第 ⌊ m×150%」 名选手的分数；所有成绩**不低于分数线**的选手全部进入面试。请划定分数线并输出进面名单。

输入 第一行两个整数 n、m（5≤n≤5000，m≤n）。随后 n 行每行两个整数：报名号（各不相同）与笔试成绩。

输出 第一行两个整数：分数线与实际进面人数。随后每行「报名号 成绩」：按成绩降序输出，成绩相同按报名号升序。

样例

输入

```
6 3
1000 90
3239 88
2390 95
7231 84
1005 95
1001 88
```

输出

```
88 5
1005 95
2390 95
1000 90
1001 88
3239 88
```

数据范围

5 ≤ n ≤ 5000，m ≤ n；题目保证 ⌊ m×150%」 ≤ n；时限 1s。

结构体排序　双关键字　模拟

时间复杂度：**O(n log n)**

思路 · 先排序，再定线放人

思路拆解

1 用结构体建模型

每位选手 =（报名号，成绩），开结构体数组读入。

2 写双关键字排序

cmp 规则：分数不同，分数高的在前；分数相同，报名号小的在前。

3 按名次定分数线

t = m×3÷2（整数除法自动向下取整），第 t 名的成绩就是分数线。

4 按分数放人

从排好序的队伍头部往下数，所有成绩 ≥ 分数线的选手都进面——同分全部放行。

一句话思路：先排序，再“按名次定线、按分数放人”。

易错提醒

- cmp 返回 true 表示 x 应排在 y 前面，先比分数再比报名号。
- m * 3 / 2 就是 ⌊ m×1.5⌋，整数除法自动向下取整。
- while 从头数成绩 ≥ line 的人，天然把同分全部放行。

用样例走一遍

排序后：1005(95) 2390(95) 1000(90) **1001(88)** 3239(88) 7231(84)

⌊ 3×1.5⌋ = 4 → 第 4 名 88 分 → 分数线 **88**

成绩 ≥ 88 的共 5 人 → 输出「88 5」

参考代码 · 分数线划定

时间复杂度 O(n log n)

```cpp
#include <iostream>
#include <algorithm>
using namespace std;

struct Player { int id, score; };    // 报名号 + 笔试成绩
Player p[5005];

bool cmp(Player x, Player y) {       // 双关键字排序规则
    if (x.score != y.score) return x.score > y.score;  // 分数高的在前
    return x.id < y.id;              // 同分：报名号小的在前
}

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < n; i++) cin >> p[i].id >> p[i].score;
    sort(p, p + n, cmp);
    int t = m * 3 / 2;               // m×150% 向下取整
    int line = p[t - 1].score;       // 第 t 名的分数就是分数线
    int cnt = 0;
    while (cnt < n && p[cnt].score >= line) cnt++;  // 实际进面人数
    cout << line << " " << cnt << endl;
    for (int i = 0; i < cnt; i++)
        cout << p[i].id << " " << p[i].score << endl;
    return 0;
}
```

代码讲解

1. 分数线是“第 t 名的分数”，不是 m×1.5 本身。
2. 同分选手全部进面，实际人数可能超过 ⌊ m×1.5⌋。
3. cmp 里同分时别忘了报名号升序，否则输出顺序错。

变式一问

如果同分时要求报名号大的在前，cmp 应该怎么改？

02

模拟测试赛二

P17 – P29 · 每题三讲：题目 / 思路 / 参考代码

A 小鱼的游泳时间
洛谷 P1425 · 入门 · 单位换算 / 数学计算

B 数字统计
NOIP 2010 普及组 · 入门 · 枚举 / 整数拆位

C 连续正整数和
洛谷 P1147 · 普及– · 枚举起点 / 累加剪枝

D 彩票摇奖
AHOI 2001 · 入门 · 桶标记 / 计数统计

小鱼的游泳时间

题目描述

小鱼从 **a 时 b 分**一直游泳到当天的 **c 时 d 分**。请你帮她算一算：这一天她一共游了多少小时多少分钟？

输入 一行四个整数 a、b、c、d，表示起止时刻（结束时刻一定晚于开始时刻）。

输出 两个整数 e 和 f，用空格隔开：一共游了 e 小时 f 分钟（f 应小于 60）。

样例

输入

```
12 50 19 10
```

输出

```
6 20
```

数据范围

$0 \le a,c \le 24$，$0 \le b,d \le 60$，结束晚于开始；时限 1s。

单位换算　数学计算

时间复杂度：**O(1)**

思路 · 统一单位再相减

一句话思路：把两个时刻统一换算成“分钟数”，相减后再还原。

思路拆解

1 识破陷阱

直接 c−a、d−b 会在 b > d 时出错（比如 0:50 到 1:10），需要借位讨论，容易漏。

2 统一单位

把两个时刻都换算成“距离 0 点 0 分多少分钟”：a×60+b 与 c×60+d。

3 相减再还原

分钟差 t 除以 60 得小时，余 60 的部分就是分钟，f = t % 60 自然小于 60。

用样例走一遍

12:50 → **770** 分，19:10 → **1150** 分

1150 − 770 = **380** 分

380 ÷ 60 = 6 余 20 → 输出 **6 20**

易错提醒

- 换算是本题唯一的“技巧”，也是所有时间题的通法。
- t / 60 得小时，t % 60 得分钟，f 自然小于 60。
- 两个整数之间用空格分隔。

测试赛二 · A 题

参考代码 · 小鱼的游泳时间

时间复杂度 O(1)

```cpp
#include <iostream>
using namespace std;

int main() {
    int a, b, c, d;
    cin >> a >> b >> c >> d;
    int t1 = a * 60 + b;          // 开始时刻换算成"第几分钟"
    int t2 = c * 60 + d;          // 结束时刻同理
    int t = t2 - t1;              // 总共游了多少分钟
    cout << t / 60 << " " << t % 60 << endl;  // 再换回 时 和 分
    return 0;
}
```

代码讲解

1. b > d 时“逐项相减”会算错，统一换成分钟最稳。
2. 输出用空格分隔，别多打换行。
3. f 一定小于 60：用 % 60 取余保证。

变式一问

如果结束时刻在第二天（跨天），换算式需要怎样修改？

数字统计

题目描述

统计给定范围 [L, R] 的所有整数中，**数字 2**一共出现了多少次。例如 [2, 22] 中：2 在数 2、12 中各出现 1 次，在 20、21 中各出现 1 次，在 22 中出现 2 次，共 6 次。

输入 一行两个正整数 L 和 R（1 ≤ L ≤ R ≤ 100000）。

输出 一个整数：数字 2 出现的次数。

样例

输入

```
2 22
```

输出

```
6
```

输入

```
2 100
```

输出

```
20
```

数据范围

1 ≤ L ≤ R ≤ 10^5；时限 1s。

枚举　整数拆位

时间复杂度：**O((R−L+1)•位数)**

思路 · 把每个数拆开看

一句话思路：“数字 2 出现几次” = 把每个数拆开一位一位看。

思路拆解

1 枚举区间内每个数

R−L+1 ≤ 10^5 个数，逐个处理毫无压力。

2 逐位拆解

x % 10 取出个位判断是不是 2；x /= 10 去掉个位继续看下一位，直到 x 变成 0。

3 计数

是 2 就 cnt++，注意 22 这样的数两位各算一次。

用样例走一遍

[2, 22]：个位是 2 → **2、12、22** 共 3 个

十位是 2 → **20、21、22** 共 3 个

合计 **6** 个 2

易错提醒

- x % 10 取个位，x / 10 去掉个位，两步一层循环。
- while (x > 0) 把每一位都检查到。
- 只在个位等于 2 时 cnt++，22 就会被数到两次。

参考代码 · 数字统计

时间复杂度 O((R−L+1)·位数)

```cpp
#include <iostream>
using namespace std;

int main() {
    int L, R, cnt = 0;
    cin >> L >> R;
    for (int i = L; i <= R; i++) {    // 枚举区间里的每一个数
        int x = i;
        while (x > 0) {               // 逐位拆解
            if (x % 10 == 2) cnt++;   // 个位是 2 就计数
            x /= 10;                  // 去掉个位，继续看下一位
        }
    }
    cout << cnt << endl;
    return 0;
}
```

代码讲解

1. 22 算两个 2：每一位单独判断，别整串只数一次。
2. while 里一定要 x /= 10，否则死循环。
3. 区间左右端点都可能含 2，从 L 开始枚举到 R，别写错循环边界。

变式一问

如果要求同时统计 0~9 每个数字的出现次数，代码应该怎么改?

连续正整数和

题目描述

给定正整数 M，求出所有的**连续正整数段**（每段至少含有两个数），使得段内全部数之和恰好等于 M。

输入 一行一个正整数 M（$10 \le M \le 2\times10^6$）。

输出 每行两个正整数：某一段的左右端点；所有段按左端点升序输出。

样例

输入

```
10000
```

输出

```
18 142
297 328
388 412
1998 2002
```

数据范围

$10 \le M \le 2\times10^6$；时限 1s。

枚举起点 **累加剪枝**

时间复杂度：**约 O(M)**

思路 · 固定起点，加过头就换

一句话思路：固定起点往后加，加过头就换下一个起点——朴素枚举 + 及时止损。

思路拆解

1 确定枚举对象

枚举段的起点 i。每段至少两个数，所以 2i+1 ≤ M 时才可能有解，起点上限约 M/2。

2 累加并剪枝

从 i 开始连续累加，一旦部分和超过 M 立即停止、换下一个起点——越加越大，继续加毫无意义。

3 恰好等于就输出

部分和 == M 时输出一段；起从小到大枚举，输出顺序自然满足左端点升序。

用样例走一遍

从起点 18 连加：18+19+…+142 = (18+142)×125÷2 = **10000** ✓

共找到 4 段：18~142、297~328、388~412、1998~2002

易错提醒

- 起点上界：i + i + 1 ≤ m，保证每段至少两个数。
- 内层一旦 sum ≥ m 就 break，这就是剪枝。
- sum 用 long long，是防爆栈的好习惯。

参考代码 · 连续正整数和

时间复杂度 约 O(M)

```cpp
#include <iostream>
using namespace std;

int main() {
    long long m, sum;
    cin >> m;
    for (long long i = 1; i + i + 1 <= m; i++) {  // 长度≥2，起点有上限
        sum = 0;
        for (long long j = i; sum < m; j++) {     // 从 i 开始连续累加
            sum += j;
            if (sum == m)           // 找到一段就输出
                cout << i << " " << j << endl;
            if (sum >= m) break;     // 超过 m 立即停止（剪枝）
        }
    }
    return 0;
}
```

代码讲解

1. “至少两个数”是硬性要求，起点 i 最多到 (M−1)/2。
2. 累加一超过 M 就要停，不剪枝容易超时。
3. 部分和用 long long 更保险（虽然本题 int 也够）。

变式一问

如果 M 最大到 10^9，枚举每个起点还行吗？能不能改成按“段长”枚举？

彩票摇奖

题目描述

彩票上印有 7 个互不相同的号码（1~33）。命中全部 7 个号码中**特等奖**，命中 6、5、4、3、2、1 个分别中一至六等奖。已知中奖号码和小明买的 n 张彩票，请统计各等奖的中奖张数。

输入 第一行自然数 n；第二行 7 个中奖号码；随后 n 行，每行 7 个号码表示一张彩票。

输出 一行 7 个整数：特等奖、一等奖至六等奖各自中奖的张数。

样例

输入

```
2
23 31 1 14 19 17 18
12 8 9 23 1 16 7
11 7 10 21 2 9 31
```

输出

```
0 0 0 0 0 1 1
```

数据范围

号码范围 1~33；n 张彩票逐张模拟即可；时限 1s。

桶标记　计数统计

时间复杂度：**O(7n)**

思路 · 对号入座

一句话思路：桶负责“查得快”，计数数组负责“记得清”。

思路拆解

1 中奖号码进桶

开标记数组 win，读到中奖号码 x 就 win[x] = true，之后查询只要 O(1)。

2 逐张彩票对号

每张彩票 7 个号码逐个查桶，命中就 match++，得到这张票命中的个数。

3 按命中数归档

开计数数组 prize，prize[match]++ 记下“中 match 个号的票有几张”。

4 按奖级输出

从 k = 7 到 1 依次输出 prize[k]，正好对应特等奖到六等奖。

用样例走一遍

中奖 18

票1：命中 **23、1** → 2 个 → 五等奖

票2：命中 **31** → 1 个 → 六等奖

输出 **0 0 0 0 0 1 1**

易错提醒

- win[x] = true 登记中奖号码，之后每次查询 O(1)。
- 每张票 7 次查桶，match 就是命中个数。
- 输出 k 从 7 到 1，正好是特等奖到六等奖。

参考代码 · 彩票摇奖

时间复杂度 O(7n)

```cpp
#include <iostream>
using namespace std;

bool win[40];                    // 桶：win[x] = 中奖号码中是否含 x
int prize[8];                    // prize[k] = 中 k 个号码的张数

int main() {
    int n;
    cin >> n;
    for (int i = 0; i < 7; i++) {
        int x;
        cin >> x;
        win[x] = true;           // 登记中奖号码
    }
    for (int i = 0; i < n; i++) {
        int match = 0;
        for (int j = 0; j < 7; j++) {
            int x;
            cin >> x;
            if (win[x]) match++;     // 命中一个中奖号码
        }
        prize[match]++;          // 按命中个数归档
    }
    for (int k = 7; k >= 1; k--) {   // 特等奖(7个) 到 六等奖(1个)
        cout << prize[k];
        if (k > 1) cout << " ";
    }
    cout << endl;
    return 0;
}
```

代码讲解

1. 号码范围 1~33，标记数组开 40 就够。
2. 命中 0 个不属于任何奖级，也不需要输出。
3. 输出顺序是特等奖在前，循环 k 从 7 到 1。

变式一问

如果还要输出每张彩票各自中了什么奖，代码应该怎么改？

03

模拟测试赛三

P30 – P42 · 每题三讲：题目 / 思路 / 参考代码

A 数字反转
NOIP 2011 普及组 · 入门 · 整数拆位 / 循环

B 多项式输出
NOIP 2009 普及组 · 普及– · 字符串构造 / 模拟

C 回文日期
NOIP 2016 普及组 · 普及– · 构造 / 合法性判定 / 枚举

D 统计单词数
NOIP 2011 普及组 · 普及– · 字符串处理 / 整词匹配

数字反转

题目描述

给定一个整数 N，请将它的各位数字反转得到一个新数。新数也要满足整数的常见形式：除非原数为 0，否则反转后的**最高位数字不能为 0**。

输入 一个整数 N（$-10^9 \le N \le 10^9$）。

输出 一个整数：反转后的新数。

样例

输入

```
123
```

输出

```
321
```

输入

```
-380
```

输出

```
-83
```

数据范围

$|N| \le 10^9$；时限 1s。

整数拆位　循环

时间复杂度：**O(位数)**

思路 · 拆位重装

思路拆解

1 符号单独处理

负数先记下符号、取绝对值，只反转数字部分。

2 循环拆位重装

ans = ans×10 + n%10：取出末位接到 ans 尾部；n /= 10 去掉末位，直到 n 变成 0。

3 末尾 0 自动消失

380 的末位 0 最先被取出，成为 ans 的最高位 0，自然舍去——083 就是 83。

一句话思路：拆位 + 重装：每次把原数的末位“搬”到答案的末尾。

易错提醒

- sign 记录符号，负数先取绝对值再拆位。
- 核心两行：ans = ans*10 + n%10，n /= 10。
- sign * ans 一行还原负号。

用样例走一遍

-380 → 符号 −，对 380 拆位

ans：0 → 8 → **83**

带上符号 → 输出 **-83**

参考代码 · 数字反转

时间复杂度 O(位数)

```cpp
#include <iostream>
using namespace std;

int main() {
    long long n, ans = 0, sign = 1;
    cin >> n;
    if (n < 0) {                    // 负号单独处理
        sign = -1;
        n = -n;
    }
    while (n > 0) {                 // 逐位反转
        ans = ans * 10 + n % 10;    // 末位接到 ans 后面
        n /= 10;
    }
    cout << sign * ans << endl;     // 末尾 0 反转后自然消失
    return 0;
}
```

代码讲解

1. 负数先取符号再拆位，否则取余会出负数。
2. 380 → 083 → 83：前导 0 自动消失，不需要特判。
3. N = 0 时循环不执行，ans = 0，恰好就是答案。

变式一问

如果输入是字符串形式的“-380”，用字符串反转该怎么做？两种方法各要注意什么？

多项式输出

题目描述

给出一个一元 n 次多项式各项的系数，按规范格式输出：次数从高到低；只含系数非 0 的项；系数为 ±1 且次数高于 0 时省略 1；次数为 1 时省略 ^1；次数为 0 时只输出系数。

输入 第一行整数 n；第二行 n+1 个整数：从 n 次项到常数项的系数（绝对值 ≤100）。

输出 一行字符串：按规范格式输出的多项式。

样例

输入

```
5
100 -1 1 -3 0 10
```

输出

```
100x^5-x^4+x^3-3x^2+10
```

输入

```
3
-50 0 0 1
```

输出

```
-50x^3+1
```

数据范围

1 ≤ n ≤ 100，|系数| ≤ 100；时限 1s。

字符串构造　模拟

时间复杂度：**O(n)**

思路 · 把格式规则翻译成 if

思路拆解

1 从高次到低次循环

i 从 n 到 0，对应“次数递减”的要求，第 i 次项系数为 c。

2 逐项四问

系数为 0？整项跳过。正还是负？非首项正数补 +。系数 ±1 且 i>0？省略 1。指数 1 或 0？分别省略 ^1 与 x。

3 首项符号特判

首项（i = n）为正时不输出 +，为负时正常输出 −。

一句话思路：输出格式题 = 把每条格式规则逐条翻译成一个 if。

易错提醒

- i 从 n 到 0，对应“次数从高到低”。
- 四个 if 依次处理四条格式规则。
- 三元表达式 c > 0 ? c : -c 取绝对值。

用样例走一遍

系数 100 −1 1 −3 0 10：

$100x^5 \to -x^4$（省略 1）$\to +x^3 \to -3x^2 \to 0$ 跳过 $\to +10$

结果 **100x^5-x^4+x^3-3x^2+10**

参考代码 · 多项式输出

时间复杂度 O(n)

```cpp
#include <iostream>
using namespace std;

int main() {
    int n, c;
    cin >> n;
    for (int i = n; i >= 0; i--) {    // 从最高次项到常数项
        cin >> c;
        if (c == 0) continue;         // 系数为 0，整项不输出
        if (c > 0 && i != n) cout << "+";  // 非首项的正数前补 +
        if (c < 0) cout << "-";
        int a = c > 0 ? c : -c;       // 系数绝对值
        if (a != 1 || i == 0) cout << a;   // ±1 且非常数项时省略
        if (i >= 1) {
            cout << "x";
            if (i >= 2) cout << "^" << i;  // 指数为 1 时省略
        }
    }
    cout << endl;
    return 0;
}
```

代码讲解

1. 首项为正不输出 +，但首项为负一定要输出 −。
2. 常数项的系数 1 不能省：省略条件是“±1 且次数高于 0”。
3. 系数 0 整项跳过，中间项同样处理。

变式一问

如果要求按指数升序输出（从常数项开始），首项与符号的判断要怎么调整?

回文日期

题目描述

日期用 8 位数字 YYYYMMDD 表示。若这 8 位数字构成回文，就称为**回文日期**。求指定起止日期之间（含端点）真实存在的回文日期的个数。

输入 两行，每行一个 8 位数字，分别表示起始与终止日期（年份 4 位，首位非 0）。

输出 一个整数：区间内回文日期的个数。

样例

输入

```
20110101
20111231
```

输出

```
1
```

输入

```
20000101
20101231
```

输出

```
2
```

数据范围

日期均为 8 位真实日期，年份 1000~9999；时限 1s。

构造　合法性判定　枚举

时间复杂度：**O(9000)**

思路 · 构造另一半

思路拆解

1 关键转化

8 位回文数完全由前 4 位（年份）决定：后 4 位 = 年份倒序。因此只需枚举年份 1000~9999，共 9000 个。

2 由年份构造月日

月份 = 年份后两位倒序，日期 = 年份前两位倒序。如 2011 → 11 月 02 日。

3 判合法再判区间

月要在 1~12，日要在 1~当月天数（2 月看闰年）；构造出的日期还要落在 [date1, date2] 内才计数。

一句话思路：不从日期枚举日期，而从“回文的前一半”构造后一半。

易错提醒

- 只枚举 9000 个年份，循环量极小。
- mm、dd 两行是“构造回文”的核心。
- 月、日、区间三层判断，缺一不可。

用样例走一遍

2011 → 倒序 **1102** → 11 月 02 日 → **20111102** 合法且在区间 ✓

2013 → 倒序 31 → 31 月？舍去

参考代码 · 回文日期

时间复杂度 O(9000)

```cpp
#include <iostream>
using namespace std;

int days[13] = {0,31,28,31,30,31,30,31,31,30,31,30,31};

bool isLeap(int y) {                // 闰年判断
    return (y % 4 == 0 && y % 100 != 0) || y % 400 == 0;
}

int main() {
    long long s, e;
    cin >> s >> e;
    int cnt = 0;
    for (int y = s / 10000; y <= e / 10000; y++) {  // 只枚举年份！
        int mm = y % 10 * 10 + y / 10 % 10;   // 月份 = 年份后两位倒序
        int dd = y / 100 % 10 * 10 + y / 1000; // 日期 = 年份前两位倒序
        if (mm < 1 || mm > 12) continue;      // 月要合法
        int maxd = days[mm];
        if (mm == 2 && isLeap(y)) maxd = 29;  // 闰年 2 月
        if (dd < 1 || dd > maxd) continue;    // 日要合法
        long long d = (long long)y * 10000 + mm * 100 + dd;
        if (d >= s && d <= e) cnt++;          // 在区间内才计数
    }
    cout << cnt << endl;
    return 0;
}
```

代码讲解

1. 老老实实逐天枚举又慢又容易错，构造法只看 9000 个年份。
2. 2 月要分闰年/平年：闰年规则是“4 的倍数且非 100 的倍数，或 400 的倍数”。
3. 构造出的月、日都可能不合法，必须验证后再计数。

变式一问

如果区间跨度固定只有一年，还能怎么枚举？构造法为什么天然不会重复计数？

统计单词数

题目描述

实现文本编辑器的查词功能：给定一个单词，输出它在文章中出现的次数和第一次出现的位置（首次出现时首字母的下标，从 0 开始）。匹配**不区分大小写**，但要求**整词完全匹配**。

输入 第一行一个单词（长度 ≤10，只含字母）；第二行一篇文章（长度 ≤10^6，只含字母和空格）。

输出 若找到：两个整数「出现次数 第一次出现的位置」；否则输出 -1。

样例

输入

```
To
to be or not to be is a question
```

输出

```
2 0
```

输入

```
to
Did the Ottoman Empire lose its power at that time
```

输出

```
-1
```

数据范围

单词 ≤ 10 字母，文章 ≤ 10^6 字符；时限 1s。

字符串处理　整词匹配

时间复杂度：**O(文章长度)**

思路 · 切词 + 整词匹配

思路拆解

1 正确读入

单词用 cin 读；随后 cin.ignore() 吃掉换行符，再用 getline 整行读入文章。

2 按空格切词

从下标 0 扫到行尾，遇到空格就得到一个单词 line[pos..i-1]，pos 是它的起点下标。

3 整词比较

把单词和每个词都转成小写，长度相等且逐字符相同才算匹配；首次匹配记下 pos，每次匹配计数。

一句话思路：“完全匹配” = 按空格切词 + 忽略大小写后整词比较。

易错提醒

- cin.ignore() 吃掉换行符，否则 getline 读到空行。
- 遇到空格就切出一个单词，pos 始终指向单词起点。
- lower() 统一小写后再整词比较；输出先次数、后位置。

用样例走一遍

文章分词：to(0) be or not to(13) be …

「To」匹配 **2 次**，首次位置 **0** → 输出 2 0

Ottoman 含 to 但不是独立单词 ✗

参考代码 · 统计单词数

时间复杂度 O(文章长度)

```cpp
#include <iostream>
#include <string>
using namespace std;

string lower(string s) {          // 全部转成小写
    for (int i = 0; i < (int)s.size(); i++)
        if (s[i] >= 'A' && s[i] <= 'Z') s[i] += 32;
    return s;
}

int main() {
    string word, line;
    cin >> word;
    cin.ignore();                 // 吃掉换行符，别让 getline 读空！
    getline(cin, line);           // 整行读入文章
    word = lower(word);
    int cnt = 0, first = -1, pos = 0; // pos: 当前单词起点下标
    for (int i = 0; i <= (int)line.size(); i++) {
        if (i == (int)line.size() || line[i] == ' ') {
            if (i > pos && lower(line.substr(pos, i - pos)) == word) {
                if (first == -1) first = pos;  // 记录首次出现位置
                cnt++;
            }
            pos = i + 1;
        }
    }
    if (cnt == 0) cout << -1 << endl;
    else cout << cnt << " " << first << endl;  // 先次数，后位置
    return 0;
}
```

代码讲解

1. 输出顺序是“次数 位置”，位置在后，别写反。
2. Ottoman 含 to 但不算：必须整词相等，不能只判断子串。
3. getline 前忘记 cin.ignore() 会读到空行，这是本题最常见的错误。

变式一问

如果单词间可能有多个连续空格，切词逻辑要怎么保证不出错？

04

共性规律总结

P43 – P49 · 每题三讲：题目 / 思路 / 参考代码

家 **规律一 · 十二题家谱**
枚举 + 模拟是同一个范式

法 **规律二 · 解题四步法**
读题建模 → 范围定法 → 步骤实现 → 边界验证

量 **规律三 · 数据范围决定算法**
10^8 次 ≈ 1 秒

器 **规律四 · 五大常用武器**
累加 / 桶 / 拆位 / 排序 / 字符串

查 **规律五 · 考场检查清单**
读题四问 + 写码五查

共性规律 · 一

十二道题，其实是一家人

同一个范式：确定枚举对象 → 逐个判定 / 统计 → 输出

流程模拟

按题意一步一步执行

- 不高兴的津津
- 小鱼的游泳时间
- 彩票摇奖

枚举 + 统计

枚举对象，再判定 / 计数

- Bovine Bones
- 珠心算测验
- 数字统计
- 连续正整数和

排序定序

结构体 + 双关键字 cmp

- 分数线划定

字符串处理

逐字符扫描 / 切词匹配

- 多项式输出
- 统计单词数

构造转化

换个更小的枚举对象

- 数字反转
- 回文日期

结论：NOIP 普及组的绝大多数题目，考的不是高级算法，而是把题意翻译成循环的功力——这正是十二道题共同的底层范式。

解题四步法：把“会做”变成流程

适用于每一道题

1 读题建模

- 圈出输入、输出与规则
- 限定词就是采分点：不同的数 / 至少两个 / 向下取整
- 把故事翻译成算式或表格

→

2 范围定法

- 看数据范围，估算运算量
- 心算标尺：10^8 次以内 ≈ 1 秒
- 决定用暴力，还是要优化

→

3 步骤实现

- 先用注释写伪代码
- 再翻译成循环与判断
- 给关键变量写注释

→

4 边界验证

- 拿样例手动演算一遍
- 检查 0、负数、并列、等号
- 确认输出格式再提交

先想清楚，再动手写——普及组的前两题，输掉的不是算法，而是题意。每道题都按这四步走一遍，错误会少一大半。

数据范围决定算法

数据范围	可接受复杂度	典型做法	本讲例证
$n \le 100$	$O(n^3)$ 也行	三重循环枚举	珠心算测验（$n \le 100$）
$n \le 5000$	$O(n \log n)$	结构体排序	分数线划定（$n \le 5000$）
$n \le 10^6$	$O(n)$	一次扫描 / 桶	数字统计、连续正整数和
组合数 $\le 10^5$	全枚举	多重循环直接枚举	Bovine Bones（≤1.6 万种）
值域 $\le 10^4$	$O(\text{值域})$	桶 / 标记数组	彩票摇奖、珠心算

心算标尺 **10^8 次以内运算 ≈ 1 秒。拿到题先估一估暴力要算多少次：本题 12 题里，没有一道需要超过 10^7 次运算——这就是“数据范围定算法”的直接体现。**

五大常用武器

新题先问：该抄哪件武器?

① 循环与累加 / 最值

求和、打擂台、逐段累加

例：津津 • 小鱼 • 连续正整数和

② 桶与标记数组

值域不大、要“查得快、记得清”

例：Bovine Bones • 彩票摇奖

③ 整数拆位

×10、%10、/10，逐位处理

例：数字统计 • 数字反转 • 回文日期

④ 结构体排序

多关键字：先比主键，再比次键

例：分数线划定

⑤ 字符串处理

逐字符扫描、切词、拼串

例：多项式输出 • 统计单词数

⑥ 组合技

一道题常常同时用几件武器

例：回文日期 = 构造 + 拆位 + 判闰年

拿到新题的顺序：先判断它属于哪一类（枚举 / 模拟 / 排序 / 字符串 / 构造），再取对应的武器，最后套四步法写完。

考场检查清单

读题四问

1. 要特判吗？0、负数、单独一个数的情况题目怎么规定？
2. 并列怎么办？取最靠前、取最小，还是都输出？
3. “不同的数”“至少两个”“向下取整”这些限定词都看到了吗？
4. 输出格式逐字对过吗？空格、换行、+ / − 号、先次数还是先位置？

写码五查

1. 会不会溢出？和、乘积接近 10^9 就换 long long。
2. 边界等号对吗？“超过 8”是 > 8，“不低于”是 ≥。
3. 循环变量都更新了吗？x /= 10 漏写就是死循环。
4. 样例手动演算过了吗？再自己造一组极端数据试试。
5. 文件名、输入输出流、endl 都检查一遍再交。

最容易丢分的 **不是不会做，而是没看见题目里的限定词。读题时把限定词圈出来，就赢了一大半。**

课后任务 · 下讲预告

把每一道“暴力题”写对，
就是普及组阶段最好的算法训练。

1. 三场比赛未 AC 的题目，按「题目 → 思路 → 代码」三步重新订正。
2. 把 12 道题按「五大武器」重新归类，写进自己的错题本。
3. 任选一道题，把思路完整讲给同桌听——能讲清楚，才是真的懂。

下讲预告：双指针与前缀和初步——从「连续正整数和」的更优解说起。